Chapter 11

Message Integrity and Message Authentication

Chapter 11

Objectives

- To define message integrity
- To define message authentication
- To define criteria for a cryptographic hash function
- To define the Random Oracle Model and its role in evaluating the security of cryptographic hash functions
- To distinguish between an MDC and a MAC
- To discuss some common MACs

11-1 MESSAGE INTEGRITY

The cryptography systems that we have studied so far provide secrecy, or confidentiality, but not integrity.

However, there are occasions where we may not even need secrecy but instead must have integrity.

Topics discussed in this section:

11.1 Document and Fingerprint
11.2 Message and Message Digest
11.3 Difference
11.4 Checking Integrity
11.5 Cryptographic Hash Function Criteria

11.1.1 Document and Fingerprint

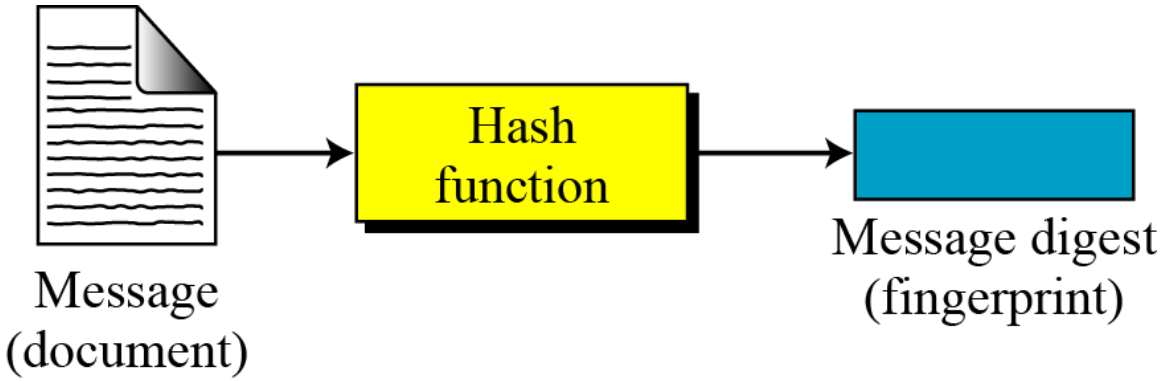
One way to preserve the integrity of a document is through the use of a fingerprint.

If Alice needs to be sure that the contents of her document will not be changed, she can put her fingerprint at the bottom of the document.

11.1.2 Message and Message Digest

The electronic equivalent of the document and fingerprint pair is the message and digest pair.

Figure 11.1 *Message and digest*



11.1.3 Difference

The two pairs (document / fingerprint) and (message / message digest) are similar, with some differences.

The document and fingerprint are physically linked together.

The message and message digest can be unlinked separately, and, most importantly, the message digest needs to be safe from change.

Note

The message digest needs to be safe from change.

hash function

A hash function accepts
a variable size message M , as input
and produces a fixed-size output
referred to as a hash code h(M) or message Digest or hash value.

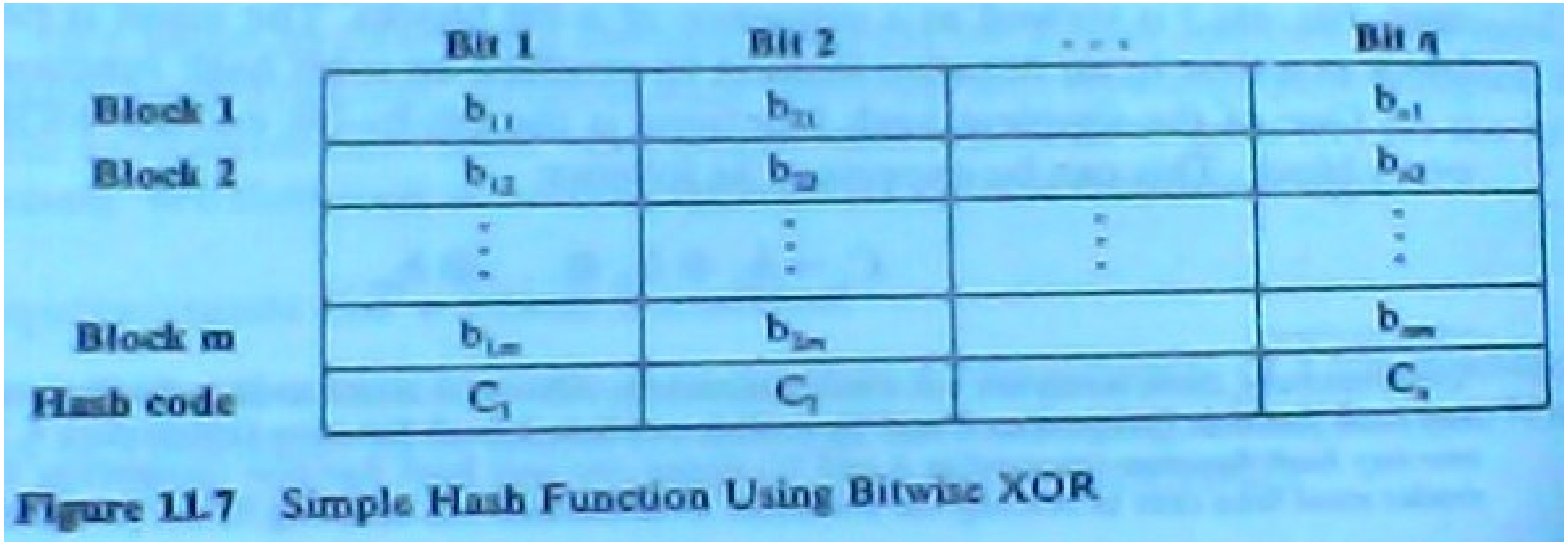
$$y = h(M)$$

-it does not use a key.
-it is only a function of input message.
- assumed to be public

A change to any bit or bits in the message, results in a change to the hash code.

Simple Hash Functions

- are several proposals for simple functions
- based on XOR of message blocks

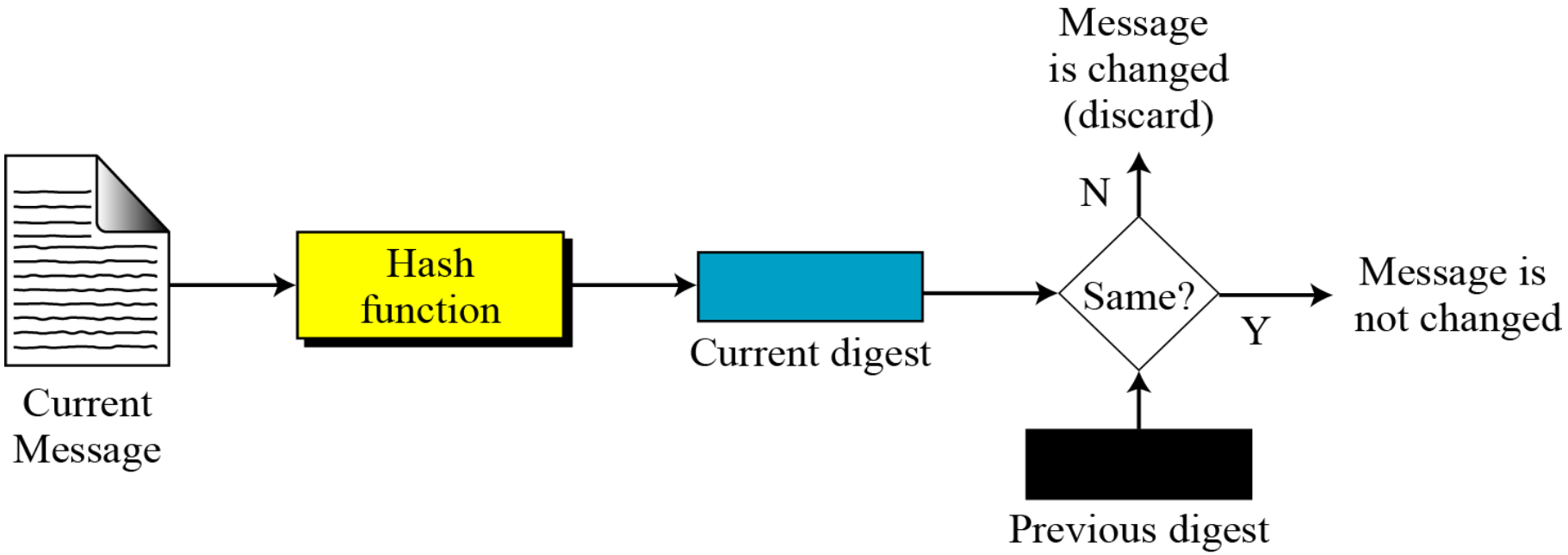


	Bit 1	Bit 2	. . .	Bit n
Block 1	b_{11}	b_{21}		b_{n1}
Block 2	b_{12}	b_{22}		b_{n2}
	⋮	⋮	⋮	⋮
Block m	b_{1m}	b_{2m}		b_{nm}
Hash code	C_1	C_2		C_n

Figure 11.7 Simple Hash Function Using Bitwise XOR

11.1.4 Checking Integrity

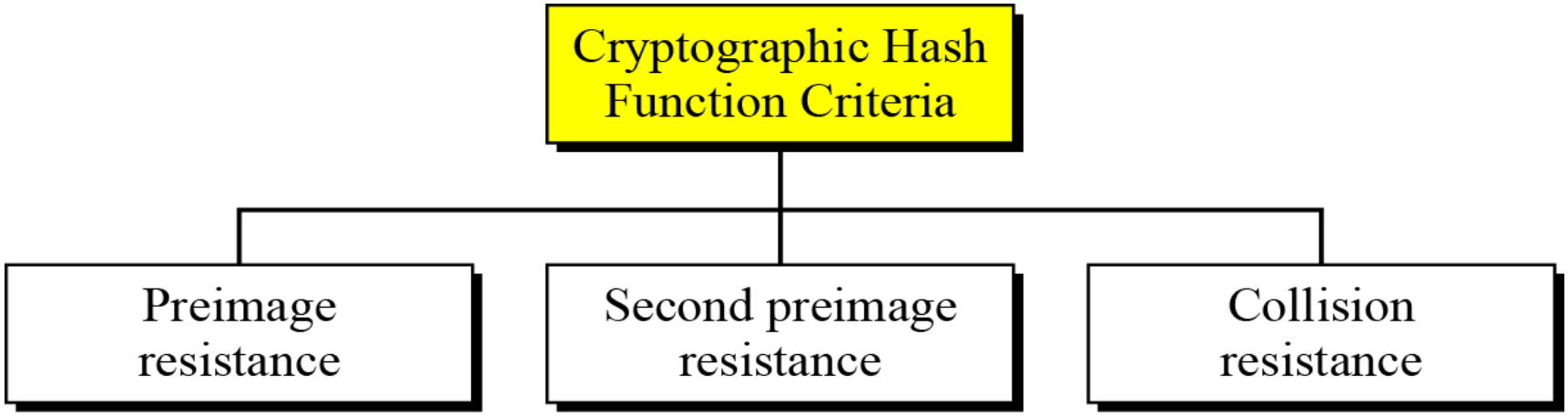
Figure 11.2 *Checking integrity*



11.1.5 Cryptographic Hash Function Criteria

A cryptographic hash function must satisfy three criteria: preimage resistance, second preimage resistance, and collision resistance.

Figure 11.3 *Criteria of a cryptographic hash function*

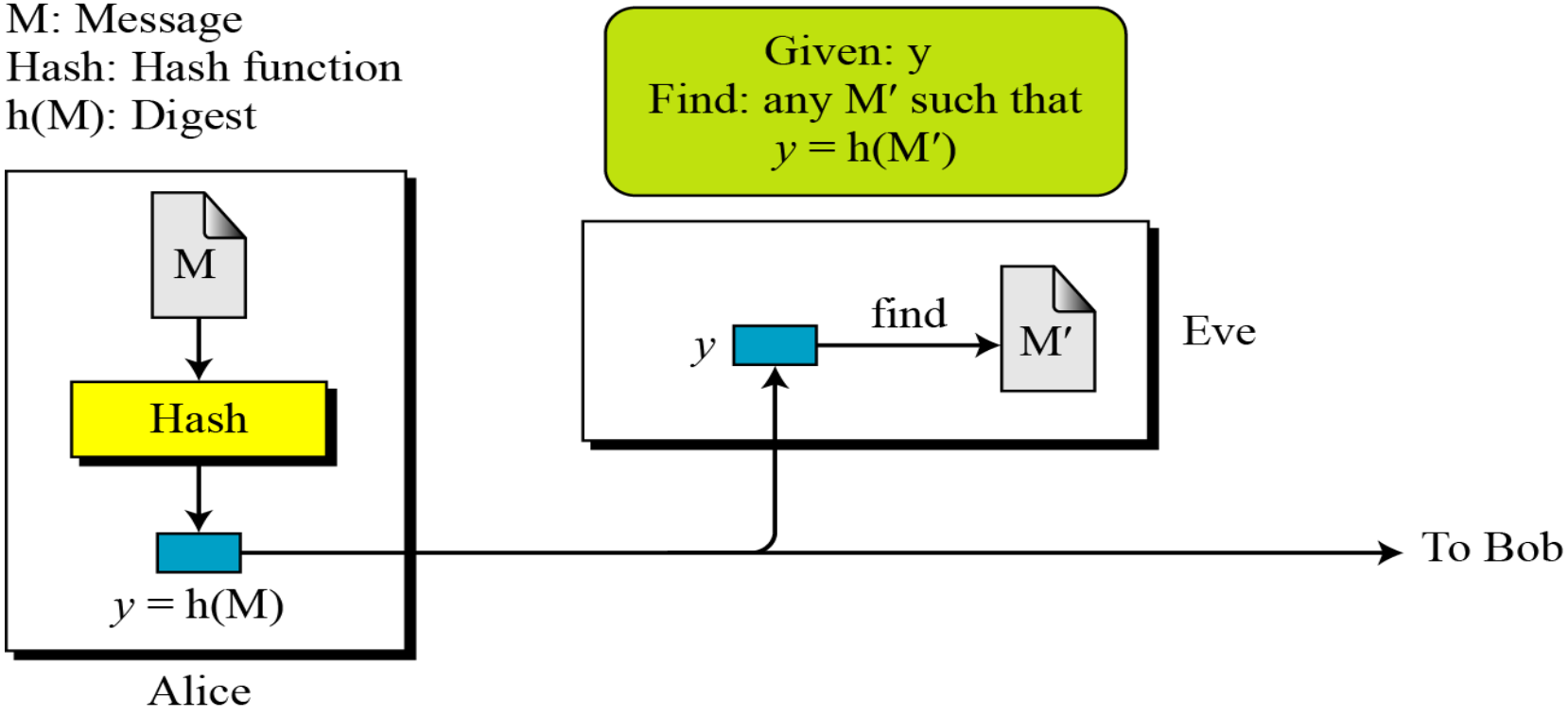


Preimage Resistance

Preimage Attack

Given: y = h(M) **Find: M′ such that y = h(M′)**

M: Message
Hash: Hash function
h(M): Digest



Example 11.1

Can we use a conventional lossless compression method such as *StuffIt* as a cryptographic hash function?

Solution

We cannot. A lossless compression method creates a compressed message that is reversible.

Example 11.2

Can we use a checksum function as a cryptographic hash function?

Solution

We cannot. A checksum function is not preimage resistant, Eve may find several messages whose checksum matches the given one.

11.1.5 Second Preimage Resistance

Ensures that a message can not easily be forged

Second Preimage Attack

Given: M and h(M) **Find: M′ ≠ M such that h(M) = h(M′)**

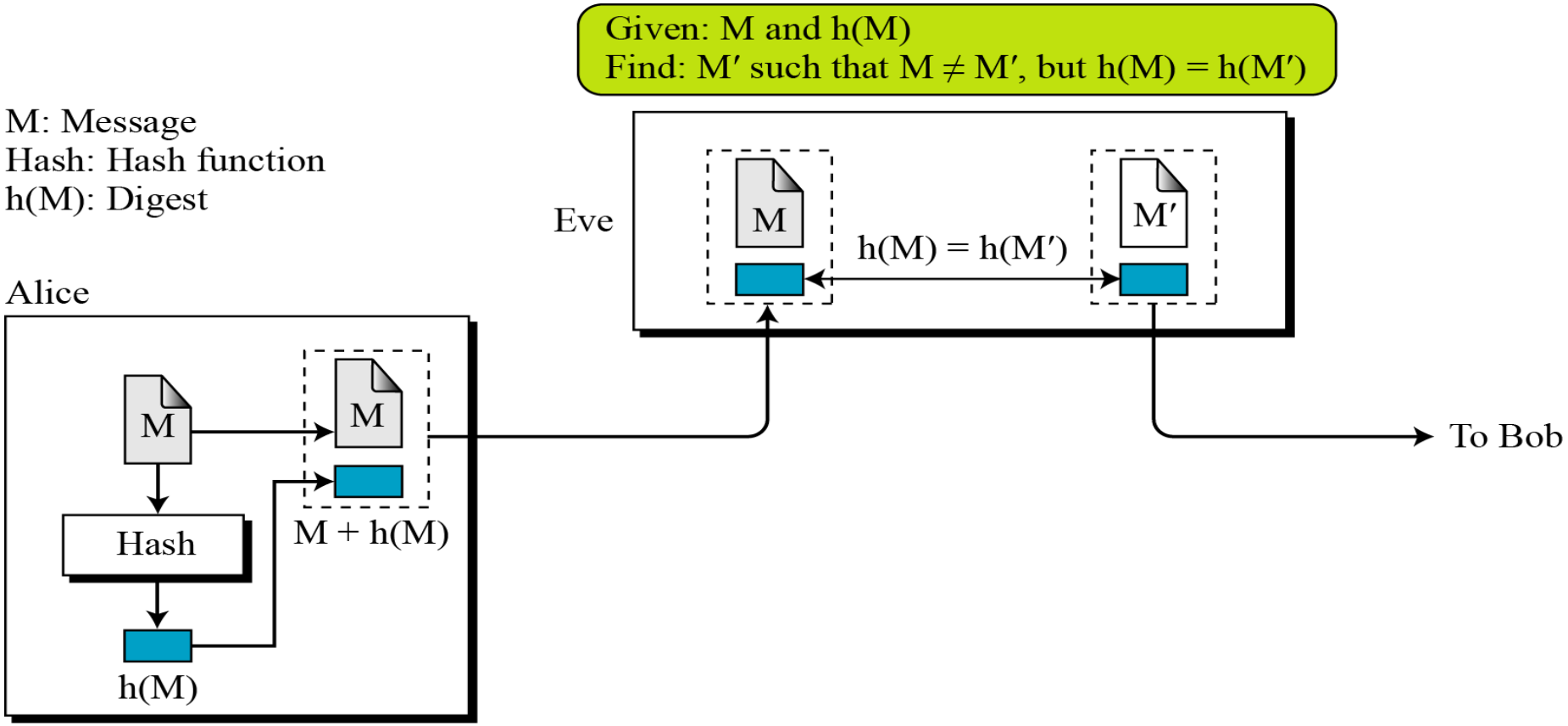


Figure 11.5 ***Second preimage***

Collision Resistance

Collision Attack

Given: none **Find: M′ ≠ M such that h(M) = h(M′)**

M: Message
Hash: Hash function
h(M): Digest

Find: M and M′ such that M ≠ M′, but h(M) = h(M′)

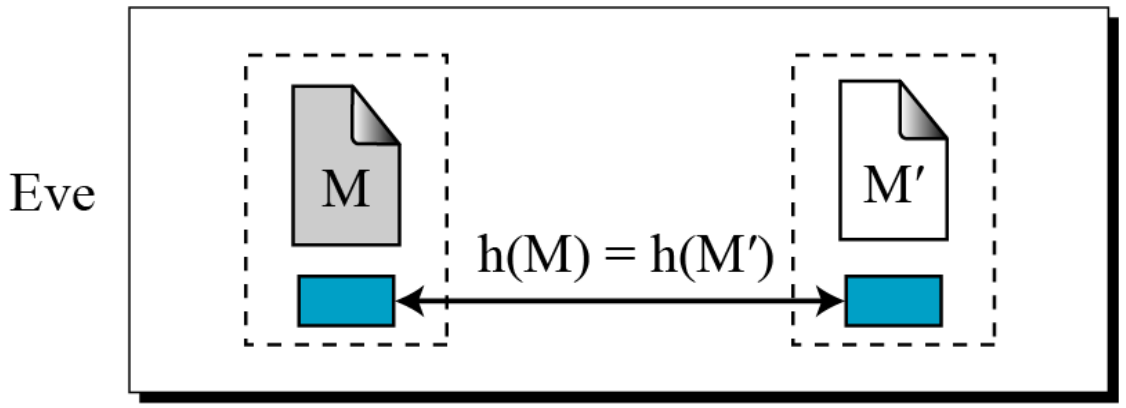


Figure 11.6 *Collision*

11-2 RANDOM ORACLE MODEL

The Random Oracle Model, which was introduced in 1993 by Bellare and Rogaway, is an ideal mathematical model for a hash function.

A random oracle is a mathematical function mapping every possible query to a random response from its output domain.

Topics discussed in this section:

11.2.1 Pigeonhole Principle
11.2.2 Birthday Problems
11.2.3 Attacks on Random Oracle Model
11.2.4 Attacks on the Structure

11-2 Continued

Example 11.3

Assume an oracle with a table. The table has two columns.

Table 11.1 *Oracle table after issuing the first three digests*

Message	*Message Digest*
4523AB1352CDEF45126	13AB
723BAE38F2AB3457AC	02CA
AB45CD1048765412AAAB6662BE	A38B

The message AB1234CD8765BDAD is given for digest calculation. The oracle checks its table. The oracle flips the coin16 times. And the result is HHTH… = 1101…

Where Head H=1 and Tail T=0

Example 11.3 Continued

Table 11.2 *Oracle table after issuing the fourth digest*

Message	*Message Digest*
4523AB1352CDEF45126	13AB
723BAE38F2AB3457AC	02CA
AB1234CD8765BDAD	DCB1
AB45CD1048765412AAAB6662BE	A38B

b. The message 4523AB1352CDEF45126 is given for digest calculation. The oracle checks its table and finds that there is a digest for this message in the table (first row). The oracle simply gives the corresponding digest (13AB).

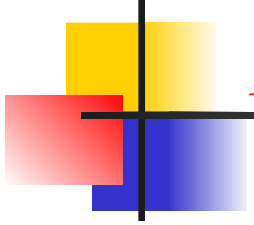
11-2 Continued

Example 11.4

The oracle in Example 11.3 cannot use a formula or algorithm to create the digest for a message. For example, imagine the oracle uses the formula h(M) = M mod n. Now suppose that the oracle has already given h(M1) and h(M2). If a new message is presented as M3 = M1 + M2, the oracle does not have to calculate the h(M3). The new digest is just [h(M1) + h(M2)] mod n since

$$h(M_3) = (M_1 + M_2) \bmod n = M_1 \bmod n + M_2 \bmod n = [h(M_1) + h(M_2)] \bmod n$$

This violates the third requirement that each digest must be randomly chosen based on the message given to the oracle.



11.2.1 Pigeonhole Principle

If n pigeonholes are occupied by n + 1 pigeons, then at least one pigeonhole is occupied by two pigeons. The generalized version of the pigeonhole principle is that if n pigeonholes are occupied by kn + 1 pigeons, then at least one pigeonhole is occupied by k + 1 pigeons.

birthday problem

What is the minimum value of k such that the probability is >= 0.5 that at least 2 people in a group of k people have the same birthday ?

i.e. find the smallest value of k
s.t. P(365,k)>=0.5

Solution:

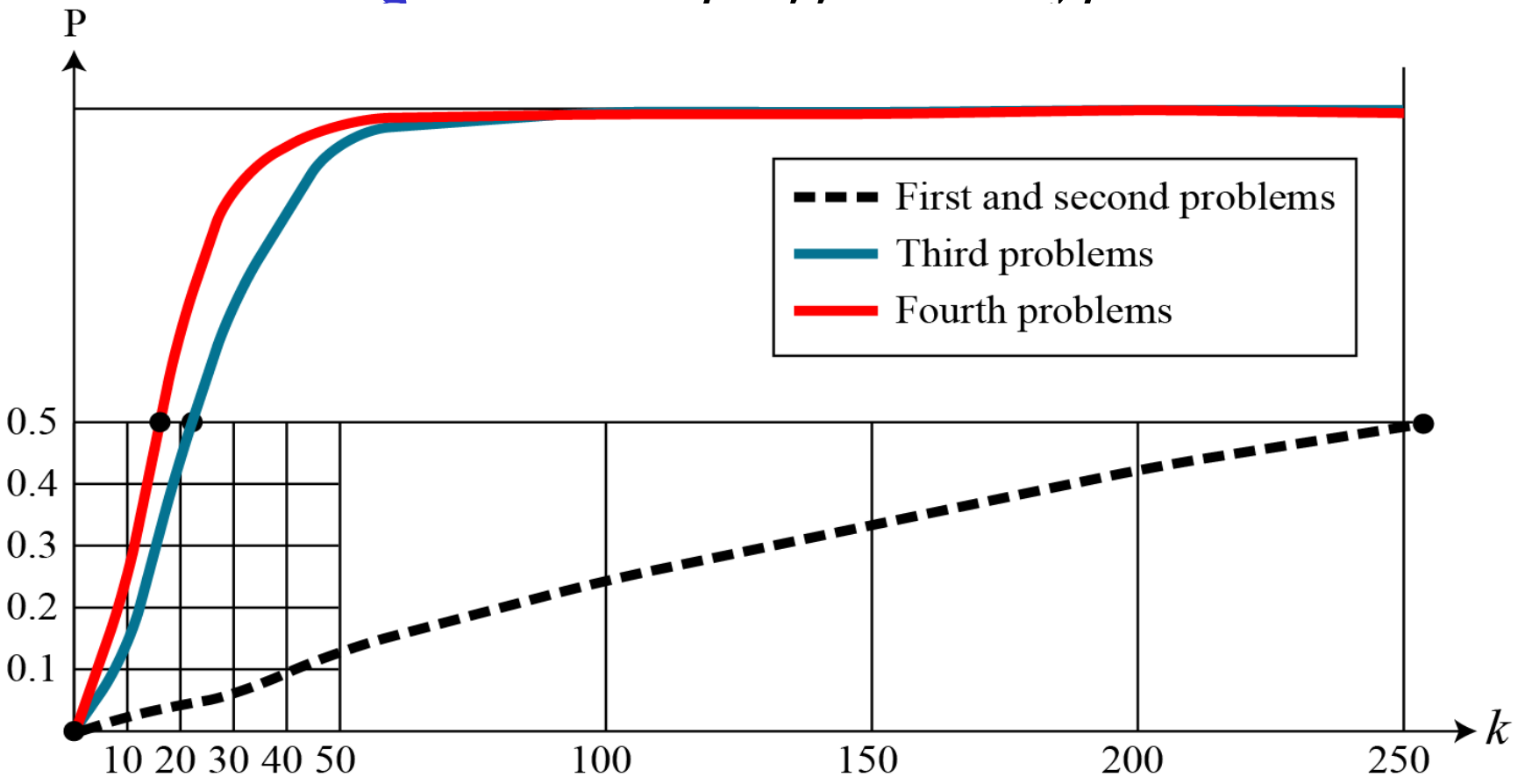
K = $\sqrt{2 \ln 2}\sqrt{N}$. = $1.18\sqrt{N}$

for n = 365

K= 1.18 x $\sqrt{365}$ = 22.54 ≈ 23

Comparison

Figure 11.8 *Graph of four birthday problem*



Example 11.6

A cryptographic hash function uses a digest of 64 bits. How many digests does Eve need to create to find the original message with the probability more than 0.5?

Solution

The number of digests to be created is $k \approx 0.69 \times 2^n \approx 0.69 \times 2^{64}$. This is a large number. Even if Eve can create 2^{30} (almost one billion) messages per second, it takes 0.69×2^{34} seconds or more than 500 years. This means that a message digest of size 64 bits is secure with respect to preimage attack, but, as we will see shortly, is not secured to collision attack.

Example 11.7

A cryptographic hash function uses a digest of 64 bits. How many digests does Eve need to create to find two messages with the same digest with the probability more than 0.5?

Solution

The number of digests to be created is $k \approx 1.18 \times 2^{n/2} \approx 1.18 \times 232$. If Eve can test 2^{20} (almost one million) messages per second, it takes 1.18×2^{12} seconds, or less than two hours. This means that a message digest of size 64 bits is not secure against the collision attack.

11-3 MESSAGE AUTHENTICATION

A message digest does not authenticate the sender of the message. To provide message authentication, Alice needs to provide proof that it is Alice sending the message and not an impostor.

The digest created by a cryptographic hash function is normally called a modification detection code (MDC).

What we need for message authentication is a message authentication code (MAC).

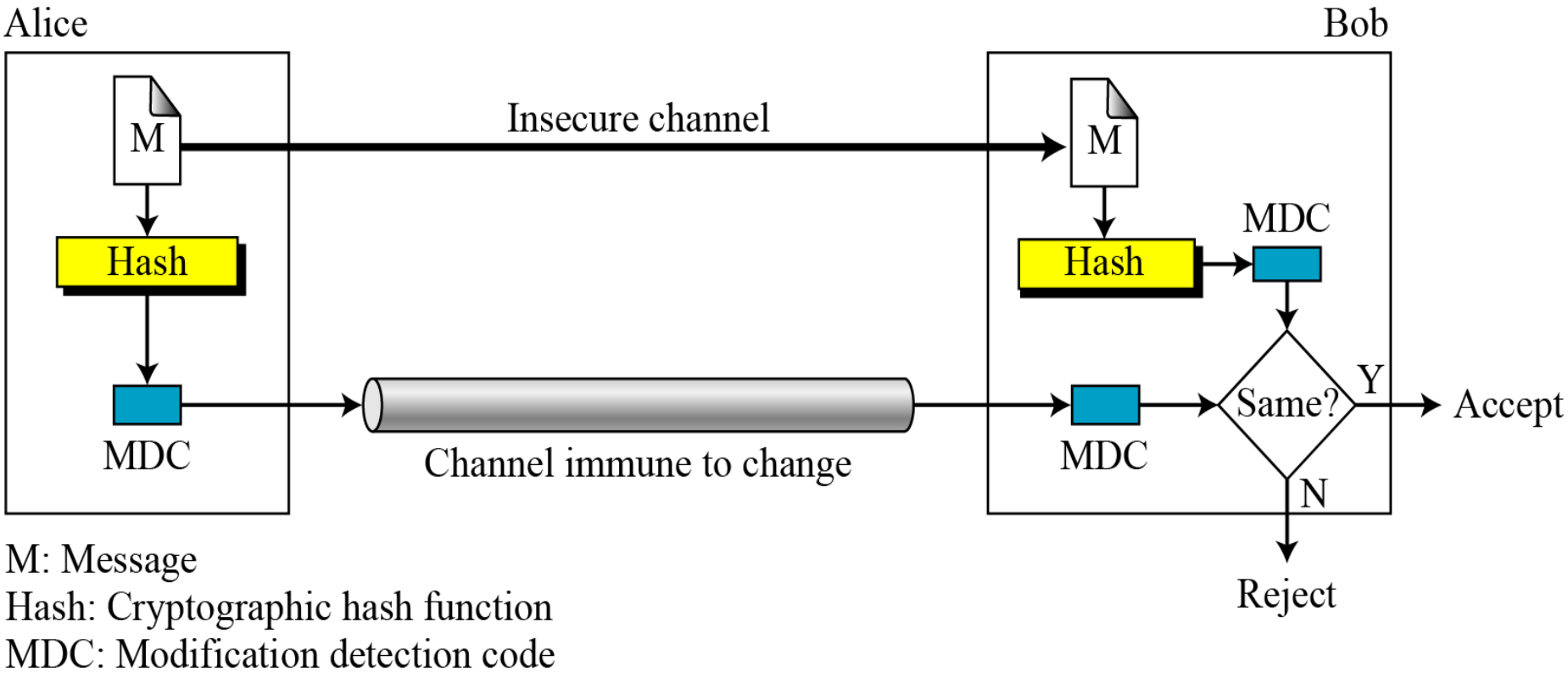
11.3.1 Modification Detection Code (MDC)

A modification detection code (MDC) is a message digest that can prove the integrity of the message: that message has not been changed.

If Alice needs to send a message to Bob and be sure that the message will not change during transmission, Alice can create a message digest, MDC, and send both the message and the MDC to Bob.

Bob can create a new MDC from the message and compare the received MDC and the new MDC. If they are the same, the message has not been changed.

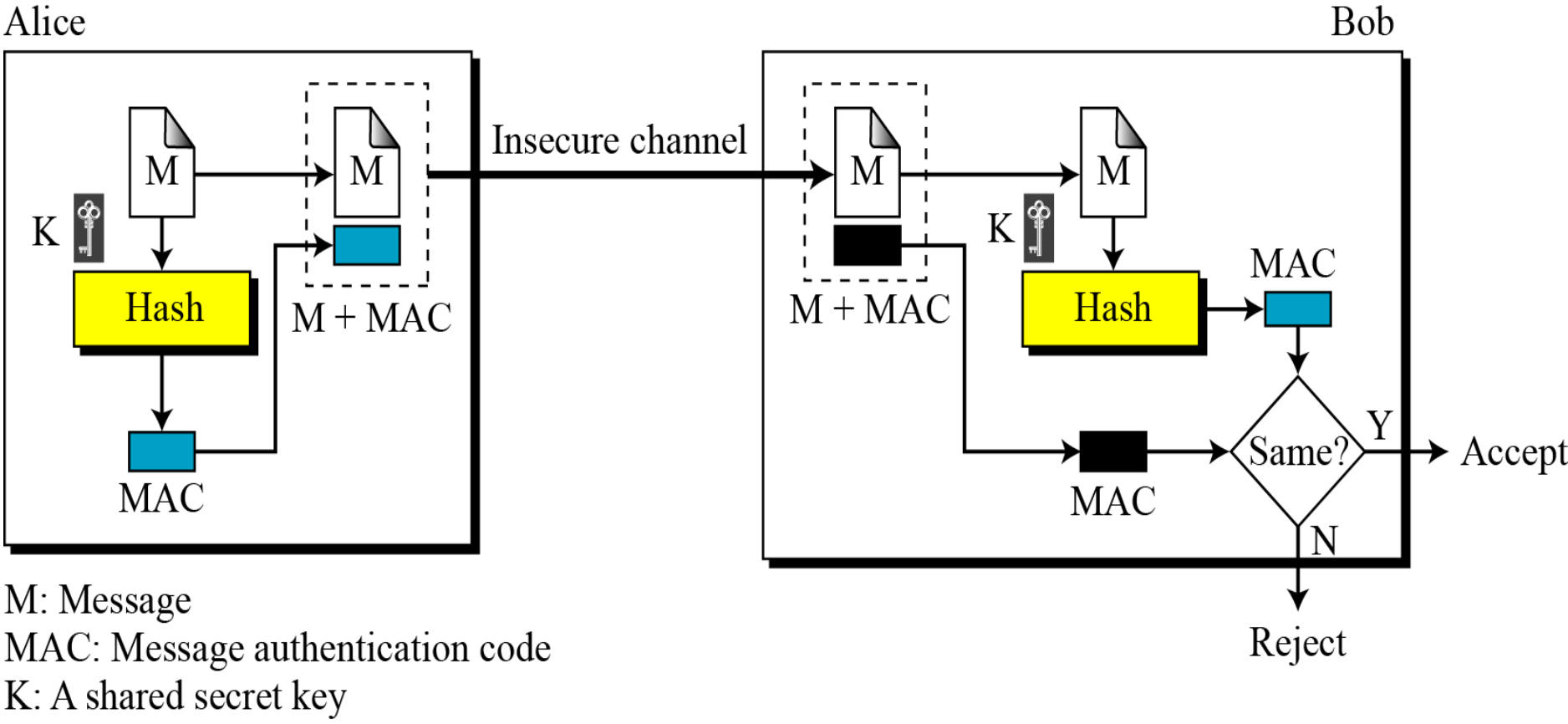
Figure 11.9 *Modification detection code (MDC)*



11.3.2 Message Authentication Code (MAC)

MAC includes a shared secret key.

Figure 11.10 *Message authentication code*



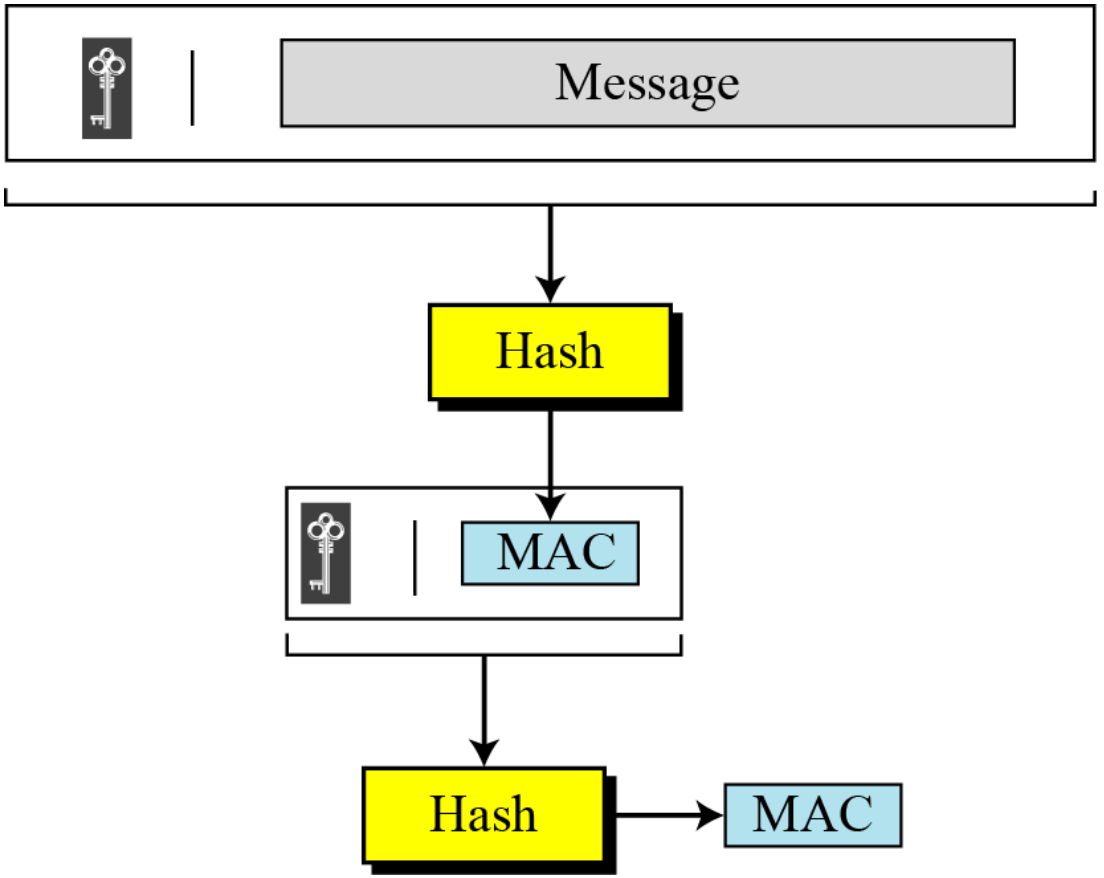
M: Message
MAC: Message authentication code
K: A shared secret key

Note

The security of a MAC depends on the security of the underlying hash algorithm.

Nested MAC

Figure 11.11 *Nested MAC*



HMAC

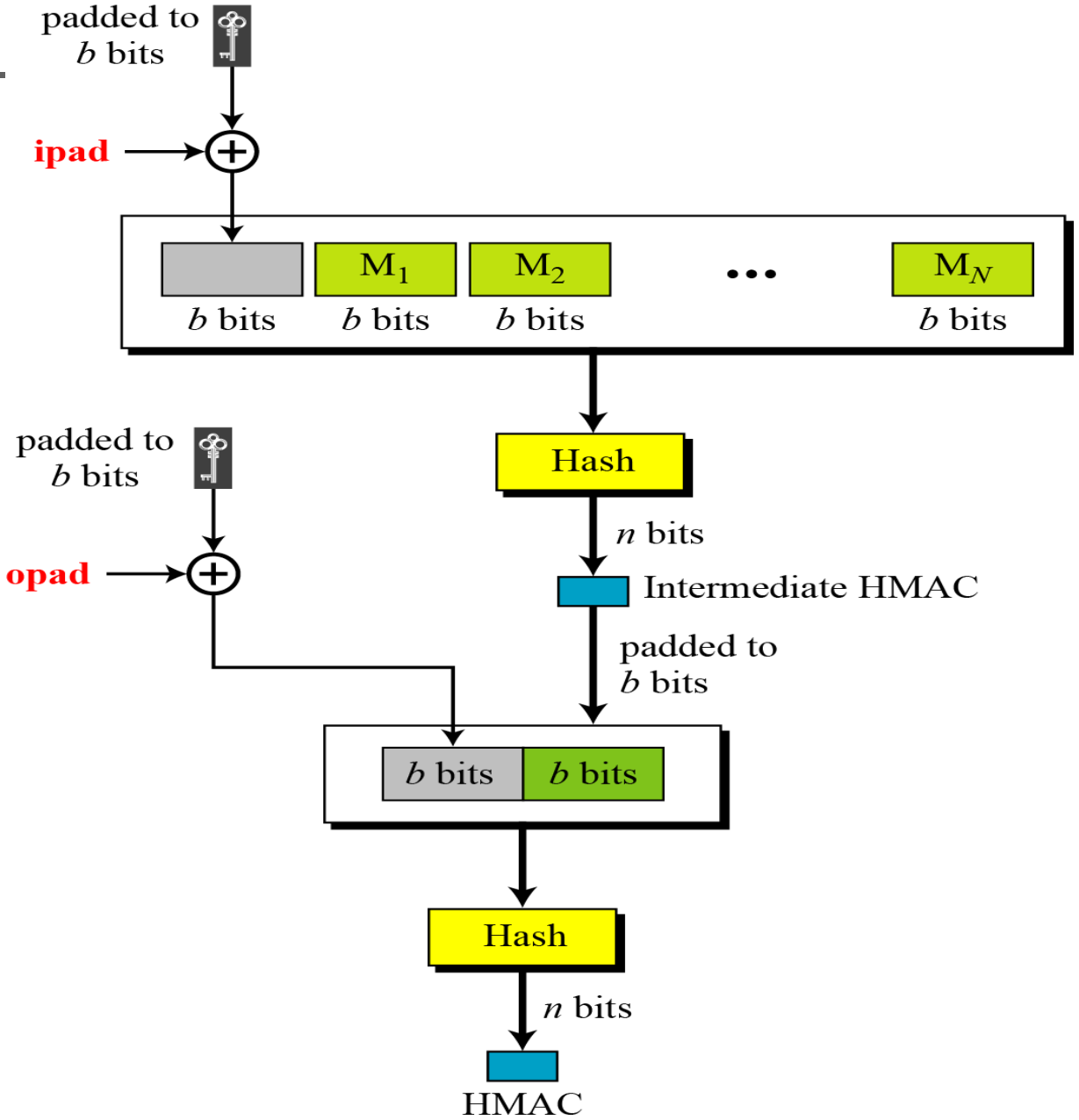


Figure 11.12
Details of Hashed MAC

Keyed Hash Message Authentication Code

$$MAC(text)_t = HMAC(K, text)_t = H((K_0 \oplus opad) || H((K_0 \oplus ipad) || text))_t$$

HMAC uses the following parameters:

B ***Block size (in bytes) of the input to the Approved hash function.***

H ***An Approved hash function.***

ipad ***Inner pad; the byte x ’36’ repeated B times.***

K ***Secret key shared between the originator and the intended receiver(s).***

L ***Block size (in bytes) of the output of the Approved hash function.***

opad ***Outer pad; the byte x’5c’ repeated B times.***

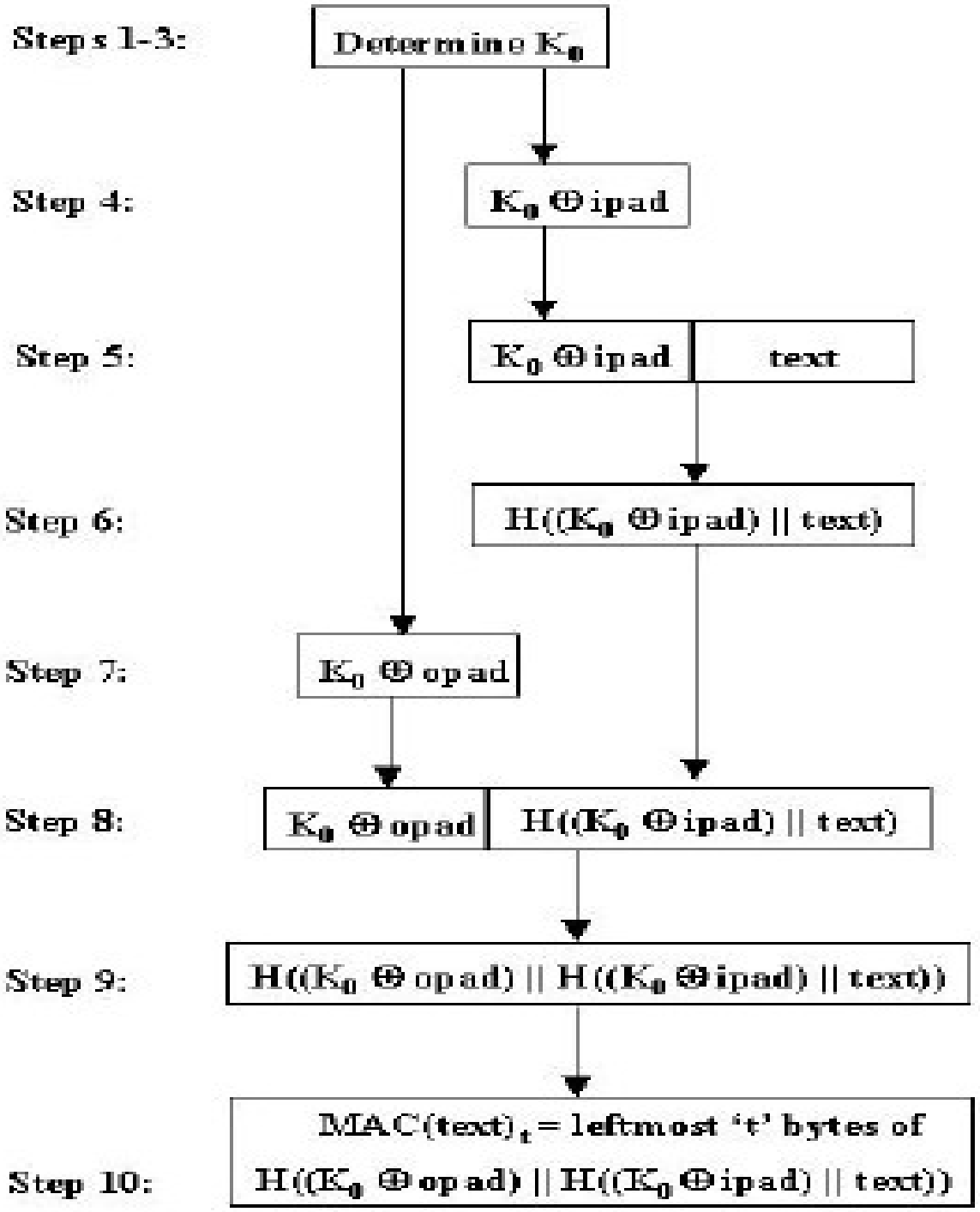
t ***The number of bytes of MAC.***

|| **Concatenation**

Å **Exclusive-Or operation**

Table 1: The HMAC Algorithm

STEPS	STEP-BY-STEP DESCRIPTION
Step 1	If the length of $K = B$: set $K_0 = K$. Go to step 4.
Step 2	If the length of $K > B$: hash K to obtain an L byte string, then append $(B\text{-}L)$ zeros to create a B-byte string K_0 (i.e., $K_0 = \mathrm{H}(K) \,\|\, 00...00$). Go to step 4.
Step 3	If the length of $K < B$: append zeros to the end of K to create a B-byte string K_0 (e.g., if K is 20 bytes in length and $B = 64$, then K will be appended with 44 zero bytes 0x00).
Step 4	Exclusive-Or K_0 with *ipad* to produce a B-byte string: $\boldsymbol{K_0 \oplus ipad}$.
Step 5	Append the stream of data '*text*' to the string resulting from step 4: $\boldsymbol{(K_0 \oplus ipad) \,\|\, text}$.
Step 6	Apply H to the stream generated in step 5: $\mathbf{H}(\boldsymbol{(K_0 \oplus ipad) \,\|\, text})$.
Step 7	Exclusive-Or K_0 with *opad*: $\boldsymbol{K_0 \oplus opad}$.
Step 8	Append the result from step 6 to step 7: $\boldsymbol{(K_0 \oplus opad) \,\|\, \mathbf{H}((K_0 \oplus ipad) \,\|\, text)}$.
Step 9	Apply H to the result from step 8: $\boldsymbol{H((K_0 \oplus opad) \,\|\, H((K_0 \oplus ipad) \,\|\, text))}$.
Step 10	Select the leftmost t bytes of the result of step 9 as the MAC.


Steps 1-3:
Determine K_0
Step 4:
$K_0 \oplus$ ipad
Step 5:
$K_0 \oplus$ ipad
text
Step 6:
H($K_0 \oplus$ ipad) || text)
Step 7:
$K_0 \oplus$ opad
Step 8:
$K_0 \oplus$ opad
H($K_0 \oplus$ ipad) || text)
Step 9:
H($K_0 \oplus$ opad) || H($K_0 \oplus$ ipad) || text))
Step 10:
$MAC(text)_t$ = leftmost 't' bytes of
H($K_0 \oplus$ opad) || H($K_0 \oplus$ ipad) || text))

11.3.2 CMAC or CBCMAC

Figure 11.13 CMAC

